AddressSanitizer
Paper Claims & Verification Plan

Figure 2: execution-time overhead | Table 1: memory usage

Prepared by: 王一涵1240012912 **Date:** 26 October 2026
Initial-submission deadline: 1 November 2026, 23:59 [5]

Submission purpose	Record the selected paper, its testable claims, and a proposed verification method for the group README or project plan. All performance measurements cited below are from the paper; no group measurements are reported.

1. Selected paper

Title: AddressSanitizer: A Fast Address Sanity Checker
Authors: Konstantin Serebryany, Derek Bruening, Alexander Potapenko, and Dmitry Vyukov.
Affiliation: Google.
Publication: 2012 USENIX Annual Technical Conference (USENIX ATC 12), pp. 309-318. [1, 2]

2. Research problem and approach

The paper studies how to detect C/C++ memory-access errors while limiting execution-time and memory overhead. AddressSanitizer (ASan) combines compiler-inserted checks with a run-time library. Shadow memory records whether application memory is accessible; redzones surround objects, and delayed reuse of freed memory helps expose use-after-free errors. The paper targets out-of-bounds accesses in heap, stack, and global objects, as well as heap use-after-free. [1, Sections 1 and 3]

3. Original evaluation conditions

Item	Conditions reported in the paper
Benchmarks	C and C++ programs from SPEC CPU2006.
Machine and OS	64-bit Linux; HP Z600; two quad-core Intel Xeon E5620 CPUs; 24 GB RAM.
Baseline	Binaries built with the regular LLVM compiler using clang -O2.
ASan configuration	32-byte heap redzones; stack unwinding on malloc/free disabled; quarantine size set to zero.
Memory measurement	VmPeak from /proc/self/status at process termination; quarantine overhead excluded.

Source: [1], Section 4, PDF p. 6. These experimental settings differ from the paper's stated run-time defaults in Section 4.3. A reproduction must record the selected settings explicitly.

Claims from Figure 2

Question: How much additional execution time does ASan require on the tested workloads?

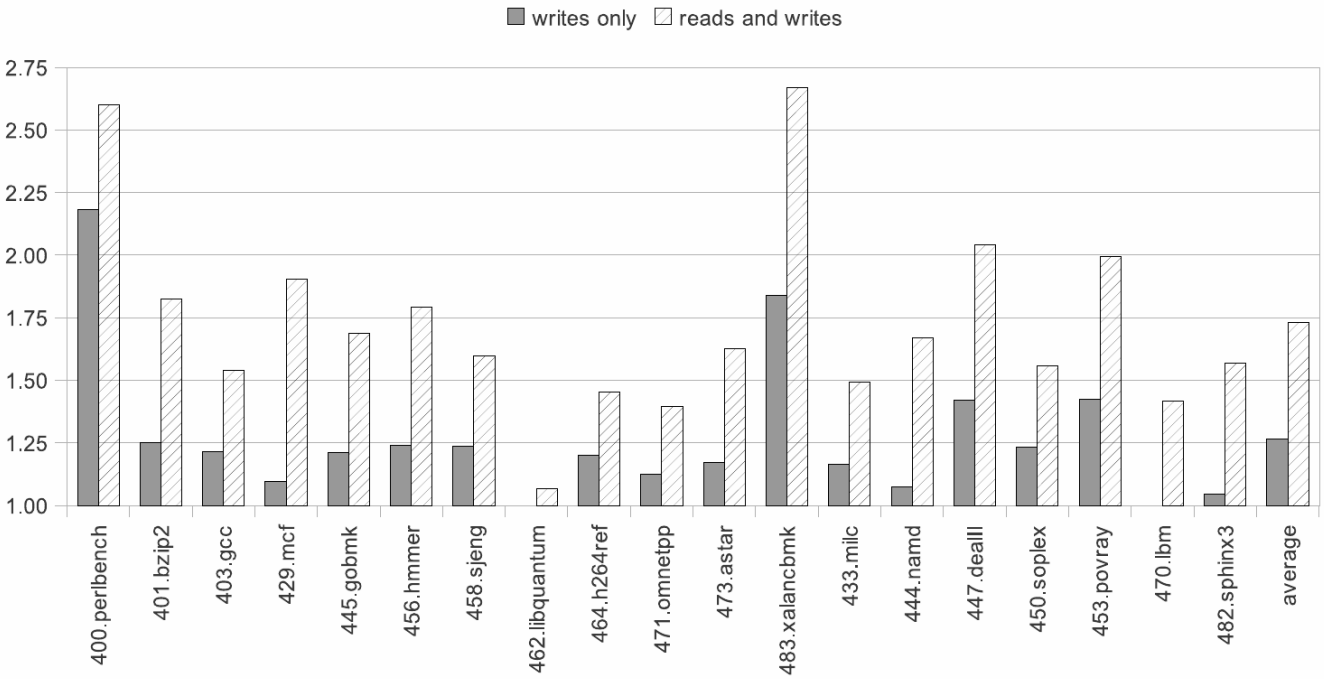


Figure 2: The average slowdown on SPEC CPU2006 on 64-bit Linux.

Original Figure 2 from [1], PDF p. 7. The bars show execution time relative to the uninstrumented baseline (1.00). This is published evidence, not a new experiment.

Claim	Published result and verification target
F2-1: Main	With reads and writes instrumented, the reported average time ratio is 1.73x, equivalent to 73% overhead. Measure the same ratio for each selected workload.
F2-2: Optional	With writes only instrumented, reported average overhead is 26% (1.26x). Include this comparison only if the chosen toolchain supports a verified writes-only mode.
F2-3: Variation	perlbench and xalancbmk show the largest reported slowdowns: 2.60x and 2.67x. The authors relate this to frequent allocation and many 1- and 2-byte accesses.

```
Runtime ratio = median(T_ASan) / median(T_baseline)
Runtime overhead (%) = (runtime ratio - 1) * 100
```

The median-based calculation is our proposed measurement method. The paper's aggregate is retained as a published result; a subset of workloads cannot independently reproduce its full-suite average. For illustration only, 10 seconds becoming 17.3 seconds means 73% overhead.

Claims from Table 1

M1: The paper reports an overall memory-usage increase of **3.37x** and attributes most of the overhead to allocator redzones. Quarantine overhead was excluded. [1, Section 4]

Table 1. Memory usage with AddressSanitizer (MB). Transcribed from [1], PDF p. 6; the reported Increase column is preserved unchanged.

Benchmark	Original (MB)	Instrumented (MB)	Reported increase
400.perlbench	670	2,168	3.64x
401.bzip2	858	1,618	2.12x
403.gcc	893	4,133	5.21x
429.mcf	1,684	2,098	1.40x
445.gobmk	37	369	11.22x
456.hmmer	33	582	19.84x
458.sjeng	180	249	1.56x
462.libquantum	104	930	10.06x
464.h264ref	72	439	6.86x
471.omnetpp	181	787	4.89x
473.astar	343	1,214	3.98x
483.xalancbmk	434	1,688	4.38x
433.milc	694	1,618	2.62x
444.namd	58	146	2.83x
447.dealII	807	2,602	3.63x
450.soplex	637	2,479	4.38x
453.povray	17	371	24.55x
470.lbm	417	550	1.48x
482.sphinx3	52	426	9.22x
total	8,171	24,467	3.37x

Arithmetic check

24,467 / 8,171 = **2.994x**, rather than the printed total of 3.37x. Likewise, 2,168 / 670 = 3.236x, whereas the perlbench row prints 3.64x. The supplied PDF does not explain the calculation that reconciles these columns. Preserve the reported figures and flag this uncertainty when comparing new results.

M2: Memory overhead varies across workloads. The reported column ranges from 1.40x (mcf) to 24.55x (povray); an aggregate must not be interpreted as the overhead of every program. [1]

For new experiments, compute memory ratios from the same metric in both versions. Keep a ratio of totals separate from an arithmetic mean of per-program ratios.

Proposed verification method

The following protocol is proposed for the group's later measurements. It does not describe completed runs.

1. Choose the workload and freeze the input. Agree with the group on source version, input files, workload size, random seed (if any), and expected output. Prefer the original benchmark where access and the environment permit; clearly label any replacement as a partial reproduction.

2. Build a controlled pair. Use the same Clang version, source, libraries, architecture, and optimization level (-O2). Build a baseline without ASan and an ASan version with -fsanitize=address at compilation and linking. Keep other flags identical and record both complete commands. [3]

3. Record the effective ASan settings. List redzone size, quarantine size, allocation stack-trace settings, and any additional enabled checks. Compare with the paper's 32-byte redzones, disabled allocation/free stack unwinding, and zero quarantine. Verify available options for the installed version; record unsupported settings or deviations. Label a run using current defaults as a separate configuration. [1, 3]

4. Check successful execution. Confirm that both versions produce the expected output and complete successfully. Save ASan reports and failed-run logs separately; do not treat an early abort as a fast benchmark result.

5. Measure time repeatedly. Proposed minimum: one unreported warm-up per version, followed by five measured runs per version in fresh processes. Alternate baseline and ASan runs, keep the machine otherwise idle, and retain every raw time. Use wall-clock seconds, report the median and range, and document any excluded run and its reason.

6. Measure and distinguish memory metrics. Use separate memory runs if the collector affects timing. Capture the measured program's own VmPeak and, for context, VmHWM near normal termination. Record the observation point, raw units, and collector. An external collector should read the benchmark's /proc/PID/status. [1, 4]

Memory comparability	VmPeak is peak **virtual** memory; VmHWM is peak **resident** memory. Current ASan maps a very large virtual address range on 64-bit platforms, so a large VmPeak does not mean equally large RAM use. Keep both measures separate and do not equate an RSS-based ratio with the paper's Table 1 metric. [3, 4]

7. Report results with their scope. For each workload, compute the time ratio and each memory ratio from paired measurements. If multiple workloads are used, state the aggregation formula and the included set. Discuss differences in workload, compiler, hardware, settings, and memory accounting before drawing a conclusion about the published numbers.

Interpretation rule

Report whether the observed trend agrees with the claim, whether its numerical magnitude is comparable, and which limitations remain. One substitute benchmark can test an overhead trend, but cannot establish the paper's 1.73x or 3.37x full-evaluation result.

References

[1] K. Serebryany, D. Bruening, A. Potapenko, and D. Vyukov. AddressSanitizer: A Fast Address Sanity Checker. Supplied paper, atc12-final39.pdf. Key locations: Section 3; Section 4 and Table 1 (PDF p. 6); Figure 2 (PDF p. 7); Section 4.3 (PDF p. 8).

[2] USENIX. Official publication record, USENIX ATC 12, 2012, pp. 309-318. usenix.org/conference/atc12/technical-sessions/presentation/serebryany

[3] LLVM Project. AddressSanitizer, Clang documentation. Usage, Flags and Options, and Limitations. clang.llvm.org/docs/AddressSanitizer.html

[4] Linux kernel documentation. The /proc Filesystem, process status fields. docs.kernel.org/filesystems/proc.html

[5] Supplied allocation screenshot, IMG_4502.png; initial submission on 1 November, 23:59. Web references [2]-[4] accessed 26 October 2026.